

班级: 232112

学号: 22375080

姓名: 杨佳宇轩

Question 1



1. 标记的第 4 行会出现编译错误

原因: 存在尚未初始化的变量 `m`

2. `m` 需要初始化才能使用, 因为 `x` 作为成员变量会被默认初始化为 `0`, 而 `m` 为局部变量, 需要初始化才可以使用

Question 2

- a.: 正确
- b.: 正确
- c.: 正确
- d.: 错误, 只有 `Overload(int m)` 和 `Overload(double m)` 是构造方法

Question 3

程序输出：

```
initialize A1
initialize A2
initialize A3
initialize A4
initialize A5
initialize A6
copy from A6
initialize B1
initialize A8
main begins
initialize A9
initialize A6
copy from A6
initialize B2
initialize A8
main ends
```

Question 4

1. 可以证明：因为初始化 **B** 实例时，先执行了静态和非静态的代码块（初始化了 **a6** 成员变量），之后才初始化了 **a8** 变量
2. 可以证明：因为对于 **B** 类中的两个非静态代码块 **A a6 = new A(6);** 与 **A a7 = new A(a6)**，程序是先执行了 **A a6 = new A(6)** 输出了 **initialize A6** 才执行 **A a7 = new A(a6)** 输出了 **copy from A6**

Question 5

在该类被第一次被加载的时候执行静态属性的初始化并且其执行顺序是按照它们在类中出现的顺序依次进行初始化。

Question 6

1. 不能

在代码类 **Initialization** 的第一行加入 **static { B.show(); }**，即在加载 **Initialization** 类的时候先执行 **B** 静态方法的访问，可以发现，**B** 类会进行加载。

```
14 class B { 5个用法
15     static A a4 = new A(4); 1个用法
16     static {
17         a4 = new A(4);
18     }
19     static A a5 = new A(5); 0个用法
20     public B(int i) { 2个用法
21         System.out.println("Initialize B" + i);
22         a8 = new A(8);
23     }
24     public static void show() { 1个用法
25         System.out.println("B has been loaded!");
26     }
27     A a7 = new A(a6); 0个用法
28 }
29
30 public class Initialization {
31     static { B.show(); }
32     static B b1 = new B(1); 0个用法
33     static B b2; 1个用法
34
35     public static void main(String[] args) {
36         System.out.println("main begins");
37         A a9 = new A(9);
38         b2 = new B(2);
39         System.out.println("main ends");
40     }
41 }
```

Run Initialization

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:J:\IntelliJ IDEA 2024.1\lib\idea_r
initialize A1
initialize A2
initialize A3
initialize A4
initialize A5
B has been loaded!
initialize A6
copy from A6
initialize B1
```

2. [带有 `static` 关键字的方法、变量、代码块] 可以调用带有 `static` 关键字的方法、变量
- [不带有 `static` 关键字的方法、变量、代码块] 可以调用都可以

Question 7

不可以，因为 `Singleton` 类的构造函数为 `private`

Question 8

因 `Singleton` 类的构造函数为 `private`，外部无法通过 `new Singleton()` 构造一个新的 `Singleton` 实例，只可以通过 `getInstance()` 来获取 `Singleton` 类被第一次加载时产生的实例，保证了这个类最多只可能存在 1 个实例同时存在

Question 9

只需要通过 `getInstance()` 方法返回的静态实例进行调用即可

The screenshot shows the IntelliJ IDEA IDE with a project named 'Lab2'. The 'Main.java' file is open, displaying a Singleton class and a Main class. The Singleton class has a private static final instance, a private constructor, a public static getInstance method, and a public void foo method. The Main class has a main method that calls Singleton.getInstance().foo(). The line 'Singleton.getInstance().foo();' is highlighted with a red rectangle. Below the code editor, the 'Run' tab is active, showing the execution output. The output includes the command to run the program and a list of initialization steps: initialize A1, initialize A2, initialize A3, initialize A4, initialize A5, B has been loaded!, initialize A6, copy from A6, and initialize B1. The status bar at the bottom indicates the file is in 'Lab2 > src > Main.java' with a line length of 12:1, LF line endings, UTF-8 encoding, and 4 spaces per tab.

```
1 class Singleton { 9 个用法
2     private static final Singleton uniqueInstance = new Singleton(); 1 个用法
3     private Singleton() { 1 个用法
4     }
5     public static Singleton getInstance() { 1 个用法
6         return uniqueInstance;
7     }
8     public void foo() { 1 个用法
9         System.out.println("Aha!");
10    }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         System.out.println("main begins...");
16         Singleton.getInstance().foo();
17     }
18 }
```

Run Initialization x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:J:\IntelliJ IDEA 2024.1\lib\idea_r

initialize A1
initialize A2
initialize A3
initialize A4
initialize A5
B has been loaded!
initialize A6
copy from A6
initialize B1

Lab2 > src > Main.java 12:1 LF UTF-8 4 个空格

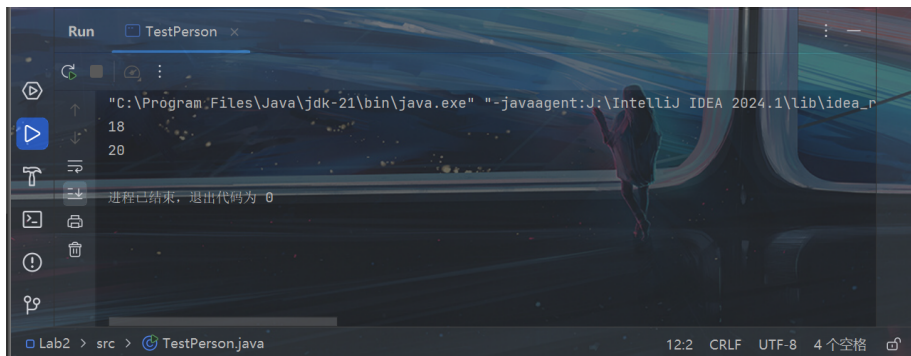
Question 10

The screenshot shows the IntelliJ IDEA IDE with a project named 'Lab2'. The 'TestPerson.java' file is open, displaying a Person class and a TestPerson class. The Person class has private fields for name, age, and sex, a private constructor, and a public constructor. The TestPerson class has a main method that creates two Person objects, sets their age, and prints their age. The status bar at the bottom indicates the file is in 'Lab2 > src > TestPerson.java' with a line length of 12:1, LF line endings, UTF-8 encoding, and 4 spaces per tab.

```
1 public class TestPerson {
2
3     public static void main(String[] args) {
4         Person person1 = new Person( name: "yiyx", age: 20, sex: "male");
5         person1.setAge(18);
6         System.out.println(person1.getAge());
7
8         Person person2 = new Person( name: "hix", age: 18, sex: "female");
9         person2.setAge(20);
10        System.out.println(person2.getAge());
11    }
12 }
13
14 class Person { 4 个用法
15     private String name; 1 个用法
16     private int age; 4 个用法
17     private String sex; 1 个用法
18
19     Person() {} 0 个用法
20     Person(String name, int age, String sex) { 2 个用法
21         this.name = name;
22         this.age = age;
23         this.sex = sex;
24     }
25 }
```

Lab2 > src > TestPerson.java 12:1 LF UTF-8 4 个空格

```
26     public void setAge(int age) { 2个用法
27         assert age >= 0 && age <= 130;
28         this.age = age;
29     }
30
31     public int getAge() { 2个用法
32         return this.age;
33     }
34
35     public void work() { 0个用法
36         System.out.println("working");
37     }
38
39     public void showAge() { 0个用法
40         System.out.println(this.age);
41     }
42 }
43
```



Question 11

```
1 import java.util.Scanner;
2
3 public class Palindrome {
4
5     @
6     public static StringBuilder reverse(String number) { 1个用法
7         StringBuilder res = new StringBuilder(-capacity: 100);
8         for(int i = number.length() - 1; i >= 0; i--) {
9             res.append(number.charAt(i));
10        }
11        return res;
12    }
13
14    public static void main(String[] args) {
15        Scanner in = new Scanner(System.in);
16        System.out.println("请输入:");
17        String number = in.nextLine();
18
19        if(number.length() == 0) {
20            System.out.println("输入长度小于1!");
21            System.exit(-status: 0);
22        }
23
24        for(int i = 0; i < number.length(); i++) {
25            if(number.charAt(i) < '0' || number.charAt(i) > '9') {
26                System.out.println("含有数字以外的字符!");
27            }
28        }
29    }
30}
```

```
26         System.exit(status: 0);
27     }
28 }
29
30 if(number.length() == 1 && number.charAt(0) == '0') {
31     System.out.println("无前导零!");
32     System.out.println("是回文数!");
33 } else {
34     StringBuilder res = reverse(number);
35     if(number.charAt(0) == '0') {
36         System.out.println("存在前导零!");
37     } else {
38         System.out.println("无前导零!");
39         if (number.contentEquals(res)) {
40             System.out.println("是回文数!");
41         } else {
42             System.out.println("不是回文数!");
43         }
44     }
45 }
46 }
47 }
```

